

Embedded Linux & System Software Interview Handbook

Chapter 6 - Device Tree & Platform Drivers (Part 4)

Topics: Platform Driver Development, probe(), remove(), Resource Management

6.7 What is a Platform Driver?

A platform driver manages on-chip peripherals such as UART, GPIO, I2C, SPI, timers and watchdogs. Unlike PCI or USB devices, platform devices are typically described by the Device Tree and matched using the compatible property.

Platform Driver Flow

Device Tree Node → Platform Device → of_match_table → probe() → Resource Initialization → Driver Ready

Typical platform_driver Structure

```
static struct platform_driver my_driver = {
    .driver = {
        .name = "my_driver",
        .of_match_table = my_of_match,
    },
    .probe = my_probe,
    .remove = my_remove,
};
```

Responsibilities of probe()

Step	Description
1	Match the Device Tree node
2	Obtain memory resources
3	Map MMIO registers
4	Request IRQ
5	Enable clocks
6	Configure GPIOs
7	Register device interfaces

Frequently Used APIs

API	Purpose
platform_get_resource()	Retrieve MMIO resource
devm_ioremap_resource()	Map registers
devm_request_irq()	Register interrupt
devm_clk_get()	Acquire clock

devm_gpiod_get()	Acquire GPIO
readl()/writel()	Access device registers

Managed Resource APIs (devm_*)

Managed APIs automatically release resources when the driver is unloaded or probe() fails. They simplify cleanup code and reduce memory leaks.

Role of remove()

remove() is invoked when the device is detached or the module is unloaded. It performs cleanup such as unregistering interfaces and stopping ongoing operations. Most resources acquired using devm_* APIs are released automatically.

Common Debugging Tips

- Verify the compatible string.
- Confirm probe() is called.
- Check dmesg for resource allocation failures.
- Validate register addresses.
- Ensure clocks and GPIOs are enabled before accessing hardware.
- Use readl()/writel() carefully to avoid invalid MMIO access.

Interview Questions

1. What is the difference between platform_device and platform_driver?
2. Why are devm_* APIs preferred?
3. What happens inside probe()?
4. When is remove() called?
5. Explain readl() and writel().
6. How do you debug a platform driver whose probe() never executes?

Key Takeaways

Platform drivers are the foundation of Linux support for SoC peripherals. Understanding probe(), resource acquisition, managed APIs and MMIO access is essential for board bring-up, BSP development and senior embedded Linux interviews.